

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO TUẦN 11
MÔN THỰC TẬP CƠ SỞ

**Đề tài: Nghiên cứu, đánh giá và triển khai
mô hình Deep Learning tối ưu cho hệ thống phân loại rác thải**

Giảng viên hướng dẫn:	TS. Kim Ngọc Bách
Sinh viên thực hiện:	Nguyễn Văn Gia Bảo
Mã sinh viên:	B23DCCN079

Hà Nội - 2026

MỤC LỤC

1. Mục tiêu công việc trong tuần	3
2. Nội dung công việc đã thực hiện	3
2.1 Phân tích và xử lý triệt để lỗi đồng bộ múi giờ (Timezone Synchronization).....	3
2.2 Thiết kế bộ lọc dữ liệu kép (Multi-criteria Filter) nâng cao UI/UX.....	4
2.3 Xây dựng cơ chế xóa kết hợp (Cascade Delete) và giải phóng bộ nhớ vật lý.....	4
2.4 Triển khai tính năng dọn dẹp toàn bộ dữ liệu tài khoản (Global Purge)	4
3. Kết quả đạt được	5
4. Khó khăn gặp phải	5
5. Kế hoạch tuần tiếp theo	7

1. Mục tiêu công việc trong tuần

Trong các tuần trước, hệ thống GreenAI đã hoàn thiện được luồng nhận diện AI và bắt đầu lưu trữ thành công tệp tin ảnh vật lý cùng các bản ghi dữ liệu người dùng. Tuy nhiên, khi hệ thống đi vào vận hành thực tế, một phần mềm chuyên nghiệp đòi hỏi khả năng quản trị vòng đời dữ liệu (Data Lifecycle Management) một cách chặt chẽ.

Chính vì vậy, mục tiêu công việc trong Tuần 11 của em tập trung hoàn toàn vào việc **Làm sạch dữ liệu (Data Cleanup), Tối ưu hóa truy vấn và Nâng cấp Trải nghiệm người dùng (UI/UX)**, cụ thể bao gồm:

- **Khắc phục lỗi logic thời gian:** Tìm ra nguyên nhân và sửa dứt điểm hiện tượng lệch múi giờ giữa thao tác quét ảnh và thao tác gửi phản hồi của người dùng.
- **Xây dựng hệ thống tra cứu thông minh:** Phát triển bộ lọc dữ liệu nhiều lớp (Lọc theo thùng rác và lọc theo khoảng thời gian) giúp người dùng tra cứu lịch sử dễ dàng khi lượng dữ liệu phình to.
- **Quản lý tài nguyên đĩa cứng an toàn:** Xây dựng tính năng "Xóa bản ghi" thông minh. Tính năng này phải giải quyết được bài toán xóa sạch cả dữ liệu dạng văn bản (text) trong cơ sở dữ liệu lẫn tệp tin đa phương tiện (image) trên ổ cứng máy chủ để ngăn chặn rò rỉ bộ nhớ (Memory/Storage Leak).
- **Kiểm soát quyền riêng tư:** Cung cấp cho người dùng quyền được dọn dẹp (reset) toàn bộ dữ liệu cá nhân của họ bằng một thao tác an toàn và dứt khoát.

2. Nội dung công việc đã thực hiện

2.1 Phân tích và xử lý triệt để lỗi đồng bộ múi giờ (Timezone Synchronization)

Trong quá trình kiểm thử, em phát hiện thời gian ở bảng "Lịch sử phản hồi" luôn bị chậm đúng 7 tiếng so với thời gian quét ảnh thực tế.

- **Nguyên nhân:** Bảng history đang được gán thời gian thực (Local Time - GMT+7) thông qua thư viện datetime của Python. Trong khi đó, bảng feedback lại sử dụng cơ chế DEFAULT CURRENT_TIMESTAMP của SQLite, tự động lấy múi giờ quốc tế (UTC - GMT+0).
- **Giải pháp:** Em đã loại bỏ cơ chế tự động của SQLite bằng cách chuyển kiểu dữ liệu của cột timestamp sang dạng TEXT. Tại hàm save_feedback(), em lập trình để Python chủ động sinh chuỗi thời gian thực datetime.now().strftime("%Y-%m-%d %H:%M:%S") và truyền thẳng vào câu

lệnh INSERT. Nhờ vậy, thời gian phản hồi được ghi nhận độc lập, chính xác và bám sát thời gian thao tác thực tế của người dùng.

- **Vá dữ liệu cũ (Data Migration):** Đối với các bản ghi đã bị lưu sai múi giờ từ các tuần trước, em đã viết một đoạn mã SQL UPDATE chạy nền một lần duy nhất để ép toàn bộ dữ liệu feedback cũ đồng bộ lại thời gian chuẩn khớp với bảng history.

2.2 Thiết kế bộ lọc dữ liệu kép (Multi-criteria Filter) nâng cao UI/UX

Để cải thiện không gian tra cứu lịch sử, em đã thay thế việc hiển thị toàn bộ dữ liệu bằng một hệ thống bộ lọc đa điều kiện trên giao diện Streamlit:

- **Bộ lọc Thùng rác:** Sử dụng st.selectbox để lọc riêng biệt rác Tái Chế, Hữu Cơ, Nguy hại hoặc Vô Cơ.
- **Bộ lọc Thời gian tách rời:** Thay vì dùng một ô lịch (calendar) gộp chung gây khó hiểu cho người dùng, em đã tách ra thành hai ô st.date_input riêng biệt: "Từ ngày" (start_date) và "Đến ngày" (end_date).
- **Thuật toán truy vấn:** Sử dụng thư viện Pandas để chuyển đổi cột chuỗi thời gian sang định dạng Date (pd.to_datetime), sau đó dùng điều kiện boolean để lọc DataFrame gốc. Nếu người dùng chọn mốc "Từ ngày" lớn hơn "Đến ngày", một cảnh báo (st.warning) sẽ được hiển thị để nhắc nhở thao tác chuẩn xác.

2.3 Xây dựng cơ chế xóa kết hợp (Cascade Delete) và giải phóng bộ nhớ vật lý

Thay vì chỉ thực thi lệnh DELETE trong CSDL, em đã xây dựng hàm delete_history_and_feedback(record_id, image_path) để xử lý rác thải kỹ thuật số:

- **Giải phóng đĩa cứng:** Khi nhấn xóa, mã nguồn sử dụng thư viện os để kiểm tra sự tồn tại của file ảnh. Nếu có, hàm os.remove(path) sẽ được gọi để xóa vĩnh viễn tệp ảnh đó khỏi máy chủ.
- **Xóa liên kết chéo:** Lệnh SQL lập tức xóa dòng lịch sử quét tương ứng, đồng thời quét luôn bảng feedback và xóa các phản hồi có chung đường dẫn image_path. Việc này đảm bảo tính toàn vẹn và sạch sẽ của CSDL.

2.4 Triển khai tính năng dọn dẹp toàn bộ dữ liệu tài khoản (Global Purge)

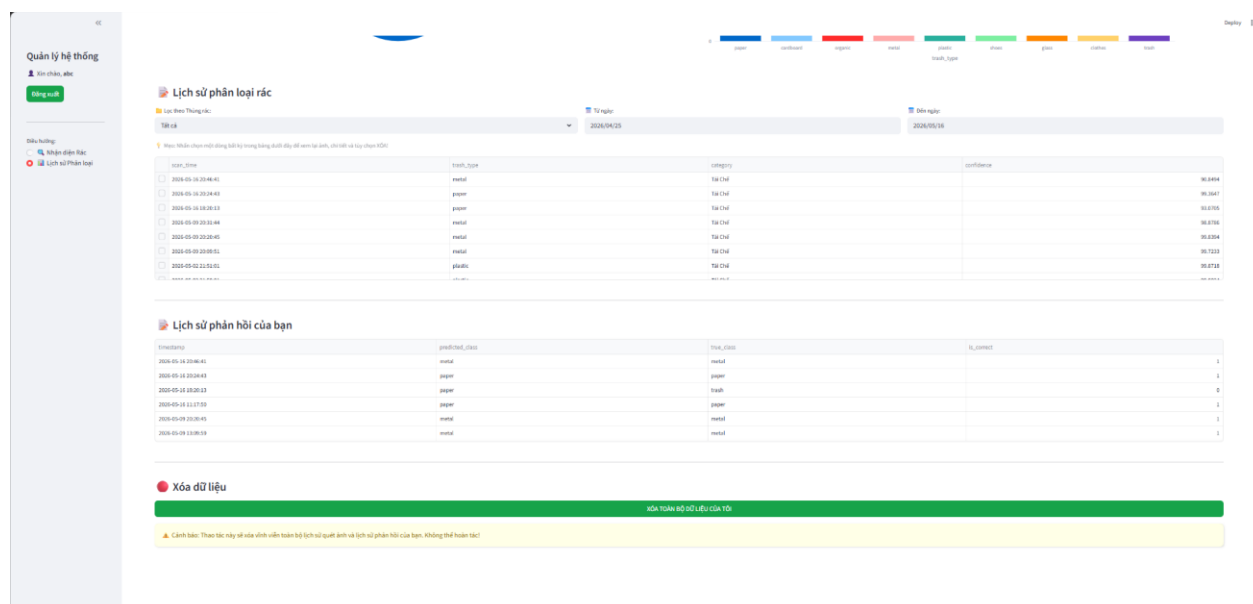
Để đảm bảo quyền riêng tư, em đặt một nút "XÓA TOÀN BỘ DỮ LIỆU CỦA TÔI" ở dưới cùng trang Dashboard kèm thông báo cảnh báo đỏ. Khi kích hoạt, hệ thống sẽ chạy vòng lặp duyệt qua toàn bộ biến df['image_path'] để xóa hàng loạt file ảnh vật lý của người dùng đó bằng hàm os.remove(), sau đó gọi lệnh SQL xóa toàn

bộ bản ghi trong cả 2 bảng history và feedback. Lệnh `st.rerun()` sẽ tự động làm mới trang web về trạng thái trông hoàn toàn.

3. Kết quả đạt được

Kết thúc Tuần 11, hệ thống đã lột xác thành một nền tảng quản trị dữ liệu hoàn chỉnh:

- **Thời gian chuẩn xác:** Xử lý thành công xung đột múi giờ giữa Python và SQLite, đảm bảo các thao tác của người dùng được ghi nhận độc lập và đúng với thời gian thực tại Việt Nam.
- **Tra cứu tốc độ cao:** Bộ lọc kép kết hợp Thùng rác - Khoảng thời gian hoạt động mượt mà, tốc độ phản hồi gần như ngay lập tức nhờ sức mạnh của thư viện Pandas.
- **Không còn Rò rỉ tài nguyên:** Luồng xóa dữ liệu từ đơn lẻ đến diện rộng được xử lý triệt để ở cả tầng vật lý (file ảnh) lẫn tầng logic (CSDL SQLite), giúp máy chủ luôn ở trạng thái tối ưu dung lượng lưu trữ.



Hình 1 - Giao diện hệ thống sau khi sửa đổi

4. Khó khăn gặp phải

Trong quá trình phát triển và hoàn thiện các tính năng nâng cao của Tuần 11, em đã đối mặt với 3 bài toán học búa liên quan đến vòng đời dữ liệu và giao diện người dùng. Dưới đây là quá trình phân tích và giải quyết vấn đề:

Khó khăn 1: Hiện tượng "Ảo ảnh dịch chuyển chỉ số" trên giao diện (UI Index Shifting Illusion)

- **Mô tả vấn đề:** Sau khi lập trình xong chức năng xóa bản ghi, em tiến hành kiểm thử. Khi em chọn xóa dòng dữ liệu đầu tiên, bản ghi đó biến mất thành công nhưng thông tin và thời gian của toàn bộ các dòng còn lại phía dưới bất ngờ bị "nhảy số", gây cảm giác dữ liệu trong CSDL đang bị sửa đổi chéo.
- **Quá trình phân tích và Khắc phục:** Em đã dùng lệnh in trực tiếp dữ liệu thô (raw data) từ SQLite ra màn hình console để đối soát. Kết quả cho thấy dữ liệu gốc hoàn toàn chính xác. Lỗi thực chất nằm ở thư viện hiển thị Streamlit: Bảng `st.dataframe` tự động render một cột số thứ tự hệ thống ngoài cùng bên trái bắt đầu từ 0, 1, 2.... Khi dòng mang nhãn số 0 bị xóa, dòng số 1 sẽ nhảy lên đầu và bị gán lại nhãn số 0. Điều này tạo ra "ảo ảnh thị giác" khiến người dùng lầm tưởng bản ghi bị biến đổi. Em đã khắc phục triệt để bằng cách cấu hình thuộc tính `hide_index=True` để ẩn cột số này đi.
- **Bài học rút ra:** Tuyệt đối không hiển thị các chỉ số kỹ thuật nội bộ của hệ thống (system indexes) ra giao diện người dùng cuối (End-user UI) để tránh gây nhầm lẫn trong trải nghiệm (UX).

Khó khăn 2: Sai lầm trong tư duy "ép giờ chéo" làm sai lệch bản chất dữ liệu (Data Integrity Trap)

- **Mô tả vấn đề:** Ban đầu, khi phát hiện lỗi lệch 7 tiếng giữa bảng lịch sử quét và lịch sử phản hồi, em đã chọn một giải pháp mang tính "đường tắt". Đó là viết một câu lệnh SQL UPDATE đặt ngay tại hàm tải trang Dashboard để ép toàn bộ thời gian của bảng phản hồi phải chạy theo thời gian của bảng quét rác.
- **Quá trình phân tích và Khắc phục:** Tuy nhiên, khi kiểm thử thực tế, em nhận thấy giải pháp này cực kỳ vô lý. Trong thực tế, một người dùng quét ảnh xong sẽ phải mất khoảng 10 đến 15 giây để đọc kết quả rồi mới quyết định bấm nút phản hồi (Đúng/Sai). Việc ép thời gian hai bảng giống hệt nhau đến từng giây làm mất đi tính chân thực của dữ liệu. Nhận ra sai lầm, em đã gỡ bỏ hoàn toàn câu lệnh ép giờ chéo đó. Thay vào đó, em can thiệp sâu vào hàm `save_feedback()`, sử dụng Python để sinh một chuỗi thời gian thực (Local Time) độc lập ngay tại đúng phần nghìn giây mà người dùng nhấn nút.

- **Bài học rút ra:** Lập trình viên phải luôn tôn trọng vòng đời khách quan của dữ liệu. Không được lạm dụng các thủ thuật xử lý giao diện bề mặt để che đậy các lỗ hổng trong logic thiết kế cơ sở dữ liệu.

Khó khăn 3: Rủi ro rò rỉ bộ nhớ vật lý (Storage/Memory Leak) khi xóa dữ liệu

- **Mô tả vấn đề:** Khi phát triển tính năng xóa, em nhận ra nếu chỉ dùng lệnh DELETE FROM history thông thường, hệ thống chỉ xóa được thông tin dạng văn bản (text) trong cơ sở dữ liệu. Các tệp tin ảnh nặng vài Megabyte vẫn nằm lại vĩnh viễn trên thư mục đĩa cứng của máy chủ.
- **Quá trình phân tích và Khắc phục:** Để dọn dẹp triệt để, em phải thiết kế một luồng xóa dữ liệu hai bước (Two-step Deletion). Trước khi gọi lệnh SQL, hệ thống phải trích xuất đường dẫn image_path, dùng thư viện os.remove() can thiệp vào tầng hệ điều hành để xóa tệp tin vật lý trước. Sau đó mới gọi SQL để xóa bản ghi chính và xóa kết hợp (Cascade Delete) các bản ghi phụ thuộc ở bảng phản hồi.
- **Bài học rút ra:** Khi làm việc với hệ thống lưu trữ đa phương tiện, khái niệm "Xóa" không chỉ nằm ở Database mà phải bao trùm cả tầng lưu trữ vật lý (File System) để đảm bảo máy chủ không bị quá tải.

5. Kế hoạch tuần tiếp theo

Định hướng Tuần 12 của em sẽ tập trung vào phân tích hiệu suất và hoàn thiện trải nghiệm cuối cùng:

- **Kiểm tra hiệu năng (Performance Profiling):** Đo lường và đánh giá thời gian phản hồi (latency) của mô hình AI khi xử lý các bức ảnh có độ phân giải lớn, từ đó đề xuất giải pháp resize ảnh trước khi lưu để giảm tải băng thông.
- **Quản trị rủi ro:** Tìm hiểu và triển khai cơ chế backup cơ sở dữ liệu trash_history.db định kỳ để phòng chống mất mát dữ liệu do sự cố hệ thống.